



UNAM · FACULTAD DE CIENCIAS

COMPLEJIDAD COMPUTACIONAL

REPORTE · PLANNING

El problema de planeación (Planning):
un problema PSPACE-completo

INTEGRANTES

Alan Alexzander González Murga	320079629
Jesus Antonio Romero Godoy	321144292

PROFESOR: Carlos Zeron Martinez

AYUDANTE: Malinali Gonzalez Lara

FECHA DE ENTREGA: 18 de noviembre de 2026

Índice

1 Introducción **3**

2 Definiciones **4**

2.1 La clase PSPACE 4

2.2 PSPACE-completitud 4

2.3 Condiciones, operadores y configuraciones 5

3 Problema de decisión Planning **7**

4 Planes de longitud exponencial y el procedimiento Path **8**

4.1 Planes de longitud exponencial 8

4.2 Cota superior sobre la longitud 9

4.3 Enfoques exponenciales 9

4.4 El procedimiento Path 9

5 Demostración de que Planning pertenece a PSPACE **11**

5.1 Correctitud 11

5.2 Espacio polinomial 11

5.3 Tiempo de ejecución 12

5.4 Conclusión: pertenencia a PSPACE 12

6 Demostración de la PSPACE-completitud de Planning **13**

6.1 PSPACE-dificultad y relación con QSAT 13

6.2 Reducción desde máquinas de Turing de espacio polinomial 13

6.2.1 Idea 13

6.2.2 Construcción 14

6.2.3 Tiempo de construcción 14

6.2.4 Correctitud 15

6.3 Conclusión: PSPACE-completitud 15

7 Ejemplo de la ejecución del algoritmo **16**

7.1 La instancia 16

7.2 La grafica implícita $\mathscr{G}$ 16

7.3 Ejecución de $\mathsf{Path}(\varnothing, M, 4)$ 16

7.4 Estado de la memoria 17

7.5 ¿Qué pasa cuando n crece? 18

8 Enfoques para resolver el problema **18**

8.1 Búsqueda explícita en $\mathscr{G}$ . . . 18

8.2 Savitch: espacio polinomial a costa de tiempo . . . 18

8.3 Planificadores prácticos . . . 18

9 Aplicaciones **19**

9.1 Logística y operaciones de socorro . . . 19

9.2 Vuelos con escalas y requisitos de visa . . . 19

9.3 Robots de almacén: un desglose completo . . . 20

9.4 Rompecabezas y juegos . . . 22

9.5 Misiones espaciales . . . 22

10 Casos especiales y variantes **22**

10.1 Longitud de plan acotada polinomialmente . . . 22

10.2 Restricciones sobre los operadores . . . 22

10.3 Planeación de primer orden . . . 23

10.4 Cuantificadores y juegos . . . 23

11 Conclusión **23**

Referencias **24**

A Código de verificación (Python) **25**

1 Introducción

Gran parte del estudio de la eficiencia algorítmica gira en torno al tiempo como recurso; así, la distinción entre P y NP descansa en esa noción. Sin embargo, la memoria es otro recurso fundamental, y al tratarla como recurso principal aparece una clase de problemas que parecen ser incluso más difíciles que NP y co-NP: la clase PSPACE (Kleinberg & Tardos, 2005, p. 531).

Uno de los problemas más representativos de PSPACE es el problema de planeación (Planning). De manera general, consiste en determinar si, partiendo de un estado inicial, existe una secuencia de acciones permitidas que permita alcanzar un estado objetivo. Es el modelo abstracto detrás de la logística de una operación de socorro, de un robot que acomoda cajas en un almacén o de los solitarios como el rompecabezas de quince (Kleinberg & Tardos, 2005, pp. 533–534). Otro ejemplo es la planeación de un viaje con escalas en el que ciertos vuelos solo pueden tomarse con documentos que se tramitan en ciertas ciudades, como una visa. Ya no basta con encontrar conexiones entre ciudades: cada acción cambia las condiciones del viajero y, con ello, las acciones que tiene disponibles (véase la sección 9.2).

Lo que hace especial a Planning es que *los planes pueden ser exponencialmente largos*, de modo que ni siquiera es posible escribir una solución completa en espacio polinomial; aun así, la pregunta de si existe un plan se puede responder usando solo espacio polinomial.

Nuestro objetivo es presentar Planning de forma autocontenida, por lo que tendremos que definirlo, demostrar que pertenece a PSPACE con el algoritmo de Savitch (1970), explicar por qué es PSPACE-completo, ejecutar el algoritmo en un ejemplo detallado, y discutir enfoques prácticos, aplicaciones y variantes.

Notación. El libro usa la misma letra $\mathscr{C}$ para el conjunto de condiciones y para las configuraciones. Aquí escribimos $c_1, \ldots, c_n$ para las condiciones, $\mathscr{C} = \{c_1, \ldots, c_n\}$ para el conjunto de condiciones y S, S', S_1, S_2 para las configuraciones (subconjuntos de $\mathscr{C}$), con S_0 la inicial y S^* la meta, para evitar ambigüedades.

2 Definiciones

2.1 La clase PSPACE

Definición 2.1 (PSPACE)**.** PSPACE es el conjunto de problemas que pueden resolverse con un algoritmo cuyo uso de espacio (memoria) es polinomial en el tamaño de la entrada (Kleinberg & Tardos, 2005, p. 531).

En tiempo polinomial un algoritmo solo puede consumir espacio polinomial, así que

$$\mathsf{P} \subseteq \mathsf{PSPACE}. \tag{1}$$

Pero PSPACE es mucho más amplia. Por ejemplo, un algoritmo que cuenta de 0 a $2^n - 1$ en base 2 solo necesita un contador de n bits (como el odómetro de un automóvil); este se ejecuta en tiempo exponencial pero usa espacio $O(n)$. La clave de aquí es que *"el espacio se puede reutilizar, mientras que el tiempo, por definición, no"* (Kleinberg & Tardos, 2005, pp. 531–532).

Teorema 2.2 (Kleinberg y Tardos, 2005, (9.2))**.** *Existe un algoritmo que resuelve 3-SAT usando solo espacio polinomial.*

Demostración. Se recorren todas las asignaciones con un contador de n bits: cuando el contador vale q, la asignación hace x_i igual al i-ésimo bit de q. Para cada asignación se evalúa la fórmula en espacio polinomial; si la satisface, se detiene, y si no, se borra el trabajo intermedio y se reutiliza ese espacio para la siguiente asignación. □

Teorema 2.3 (Kleinberg y Tardos, 2005, (9.3))**.** $\mathsf{NP} \subseteq \mathsf{PSPACE}$.

Demostración. Si $Y \in \mathsf{NP}$, entonces $Y \leq_{\mathrm{P}}$ 3-SAT: hay un algoritmo para Y que usa un número polinomial de pasos y de llamadas a una caja negra para 3-SAT. Implementando la caja negra con el algoritmo del Teorema 2.2, se obtiene un algoritmo para Y en espacio polinomial. □

Como PSPACE es cerrada bajo complemento, también co-NP $\subseteq$ PSPACE. Además, PSPACE $\subseteq$ EXPTIME, porque un algoritmo en espacio polinomial solo puede visitar un número exponencial de configuraciones distintas (Sipser, 2013, cap. 8). La Figura 1 resume las contenciones; no se sabe demostrar que sean estrictas.

2.2 PSPACE-completitud

Definición 2.4 (PSPACE-completo)**.** Un problema X es PSPACE-completo si (i) $X \in \mathsf{PSPACE}$ y (ii) para todo $Y \in \mathsf{PSPACE}$ se cumple $Y \leq_{\mathrm{P}} X$ (Kleinberg & Tardos, 2005, p. 533).

Igual que en NP, si Y es PSPACE-completo y $X \in \mathsf{PSPACE}$ cumple $Y \leq_{\mathrm{P}} X$, entonces X también es PSPACE-completo (Kleinberg & Tardos, 2005, p. 544).

El problema "raíz" es QSAT (3-SAT cuantificado), cuya completitud demostraron Stockmeyer y Meyer (1973).

Por el teorema de Savitch (1970), NPSPACE = PSPACE (Sipser, 2013, teorema 8.5), lo cual permite trabajar indistintamente con máquinas deterministas o no deterministas de espacio polinomial.

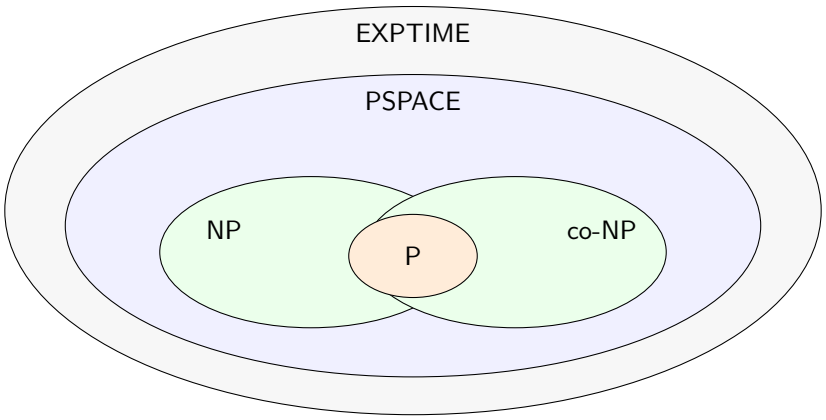


Figura 1: Relaciones entre clases (basada en Kleinberg & Tardos, 2005, Fig. 9.1).

2.3 Condiciones, operadores y configuraciones

El modelo de Planning es una versión proposicional del formalismo STRIPS (Fikes & Nilsson, 1971), tal como lo presentan Kleinberg y Tardos (2005, pp. 533–534, 538–539).

Definición 2.5 (Modelo de planeación)**.** Un modelo de planeación consta de lo siguiente.

- Un conjunto de *condiciones* $\mathscr{C} = \{c_1, \ldots, c_n\}$.
- Una *configuración* o *estado* es un subconjunto $S \subseteq \mathscr{C}$ que representa las condiciones que se cumplen en un momento dado.

  Cada condición puede ser verdadera o falsa, por lo que existen 2^n configuraciones posibles, correspondientes a todas las combinaciones de valores verdaderos y falsos de las n condiciones.
- Un conjunto de *operadores* o *acciones* $\{O_1, \ldots, O_k\}$. Cada O_i tiene una *lista de prerrequisitos* $P_i \subseteq \mathscr{C}$, una *lista de adición* $A_i \subseteq \mathscr{C}$ y una *lista de eliminación* $D_i \subseteq \mathscr{C}$.
- O_i *es aplicable* en S si $P_i \subseteq S$. Al aplicarlo se obtiene la configuración

$$S' \;=\; (S \setminus D_i) \cup A_i.$$

  Se escribe $S \xrightarrow{O_i} S'$. O_i puede aplicarse aunque haya otras condiciones además de P_i, y no afecta las condiciones que no están en A_i ni en D_i (Kleinberg & Tardos, 2005, p. 538).

Observación 2.6. El libro no precisa qué ocurre si una condición está a la vez en A_i y en D_i. Aquí se adopta la convención $S' = (S \setminus D_i) \cup A_i$ (primero se elimina y luego se agrega); en las construcciones de este reporte las listas A_i y D_i siempre se toman disjuntas, de modo que la convención no importa.

Ejemplo 2.7 (Rompecabezas de quince)**.** Según el libro, se modela con una condición por cada posible ubicación de cada ficha y un operador por cada movimiento de una ficha entre dos ubicaciones adyacentes; el prerrequisito es que una ubicación contenga a la ficha y la otra al hueco (Kleinberg & Tardos, 2005, p. 534).

Nótese que el modelo no tiene condiciones negativas: para expresar "la casilla c está libre" hace falta una condición positiva explícita, como "el hueco está en c".

Se usará el mismo recurso en la aplicación de la sección 9.

3 Problema de decisión Planning

Planning

Entrada: un conjunto de condiciones $\mathscr{C} = \{c_1, \ldots, c_n\}$, un conjunto de operadores o acciones $O_1, \ldots, O_k$ con sus respectivas listas de prerrequisitos, adición y eliminación (P_i, A_i, D_i), una configuración inicial $S_0 \subseteq \mathscr{C}$ y una configuración meta $S^* \subseteq \mathscr{C}$.

Pregunta: ¿existe una secuencia de operadores $O_{i_1}, \ldots, O_{i_m}$ tal que, empezando en S_0, cada uno sea aplicable y se llegue exactamente a S^* (es decir, se cumplen precisamente las condiciones de S^* y ninguna otra)?

El tamaño de la entrada es polinomial en n, k y la longitud de las listas; en particular, es al menos n.

Además, como los k operadores se describen explícitamente, el tamaño de la entrada también es al menos proporcional a k.

Un *plan* es una secuencia de operadores que permite llegar desde la configuración inicial hasta la meta.

Sin embargo, Planning es un problema de decisión, por lo que únicamente nos interesa determinar si existe dicho plan, sin necesidad de construirlo o almacenarlo completo.

Observación 3.1 (Una grafica implícito). Se puede representar una instancia de Planning mediante una grafica dirigido $\mathscr{G}$, donde cada nodo representa una de las 2^n configuraciones posibles y existe una arista de S a S' si algún operador permite pasar de una configuración a otra en un solo paso.

De esta manera, Planning equivale a preguntar si *existe un camino en* $\mathscr{G}$ *desde* S_0 *hasta* S^* (Kleinberg & Tardos, 2005, p. 539).

Aunque $\mathscr{G}$ puede tener un número exponencial de nodos, está descrito de forma compacta mediante n condiciones y k operadores.

Esto significa que no necesitamos almacenar todas las configuraciones ni todas las aristas para conocer las transiciones posibles, ya que podemos calcularlas a partir de los operadores.

Por lo tanto, construir explícitamente la grafica completa puede requerir espacio exponencial, lo que motiva la búsqueda de un algoritmo capaz de decidir si existe un camino utilizando únicamente espacio polinomial.

4 Planes de longitud exponencial y el procedimiento Path

Antes de demostrar la complejidad de Planning, es necesario entender por qué los planes pueden ser tan largos y por qué los algoritmos tradicionales de búsqueda pueden requerir demasiada memoria. Después presentaremos el procedimiento Path, basado en la técnica de Savitch, que permite decidir si existe un plan sin almacenarlo completo.

4.1 Planes de longitud exponencial

En algunas instancias hay planes cortos (como en el rompecabezas de quince), pero esto no ocurre en general.

Teorema 4.1 (Kleinberg y Tardos, 2005, (9.6))**.** *Existen instancias de* Planning *con n condiciones y n operadores que tienen solución, pero cuyo plan más corto tiene longitud* $2^n - 1$.

Demostración. La instancia codifica el incremento de un contador de n bits desde todos ceros hasta todos unos.

- Condiciones $c_1, \ldots, c_n$ y operadores $O_1, \ldots, O_n$.
- O_1 no tiene prerrequisitos ni lista de eliminación; solo agrega c_1.
- Para $i > 1$, O_i requiere c_j para todo $j < i$; al invocarse agrega c_i y elimina c_j para todo $j < i$.
- $S_0 = \varnothing$ y $S^* = \{c_1, \ldots, c_n\}$.

Se afirma, por inducción sobre i, lo siguiente: *desde cualquier configuración que no contenga ningún* c_j *con* $j \leq i$*, existe una secuencia de operadores que llega a una configuración que contiene a todos los* c_j *con* $j \leq i$*, y toda secuencia así tiene al menos* $2^i - 1$ *pasos.*

Para $i = 1$ es claro. Para $i > 1$ hay una solución: (1) por inducción se logran $\{c_{i-1}, \ldots, c_1\}$ con $O_1, \ldots, O_{i-1}$; (2) se invoca O_i, que agrega c_i y elimina todos los demás; (3) por inducción se logran de nuevo $\{c_{i-1}, \ldots, c_1\}$, y c_i se conserva.

Para la cota inferior, considérese el primer momento en que se agrega c_i. Solo O_i puede agregarla, y para invocarlo deben estar presentes $c_{i-1}, \ldots, c_1$, lo que por inducción requirió al menos $2^{i-1} - 1$ pasos. Pero O_i elimina todos los c_j con $j < i$, así que hay que lograrlos otra vez, lo que requiere otros $2^{i-1} - 1$ pasos.

En total, al menos $2(2^{i-1} - 1) + 1 = 2^i - 1$ pasos. Aplicando la afirmación con $i = n$ se obtiene el resultado. □

Ejemplo 4.2 ($n = 3$)**.** El plan más corto tiene 7 pasos: $O_1, O_2, O_1, O_3, O_1, O_2, O_1$, que recorre las configuraciones

$$\varnothing \to \{c_1\} \to \{c_2\} \to \{c_1, c_2\} \to \{c_3\} \to \{c_1, c_3\} \to \{c_2, c_3\} \to \{c_1, c_2, c_3\}.$$

(Es exactamente la cuenta en binario con el bit menos significativo a la izquierda.) Este plan fue verificado con el programa del apéndice A.

4.2 Cota superior sobre la longitud

Teorema 4.3 (Kleinberg y Tardos, 2005, (9.7))**.** *Si una instancia de* Planning *con n condiciones tiene solución, entonces tiene una con a lo más* $2^n - 1$ *pasos.*

Demostración. La grafica $\mathscr{G}$ tiene 2^n nodos. Si hay un camino de S_0 a S^*, el más corto no visita ningún nodo más de una vez; por tanto, tiene a lo más $2^n - 1$ aristas. □

Observación 4.4 (Consecuencia: el plan no sirve como certificado). Si todo plan tuviera longitud polinomial, bastaría exhibirlo y Planning estaría en NP. Pero el Teorema 4.1 muestra que el plan más corto puede ser exponencial, así que no es un buen certificado.

Además, en espacio polinomial ni siquiera se puede almacenar una representación explícita de la solución (Kleinberg & Tardos, 2005, p. 540). La pregunta es entonces si se puede *decidir* la existencia de un plan sin tener nunca la solución completa a la vista.

Esto no demuestra que Planning esté fuera de NP, sino que una secuencia explícita de acciones no siempre constituye un certificado de tamaño polinomial.

4.3 Enfoques exponenciales

Un primer algoritmo construye $\mathscr{G}$ y aplica búsqueda en profundidad (DFS) o en amplitud (BFS) desde S_0. Esto toma espacio exponencial solo para construir la grafica.

Se podría simular DFS o BFS sin construir $\mathscr{G}$, pero tampoco es viable: DFS debe mantener la lista de nodos del camino actual y BFS la de la frontera, y ambas pueden crecer de forma exponencial (Kleinberg & Tardos, 2005, p. 541).

Hace falta un algoritmo de búsqueda de caminos fundamentalmente distinto.

4.4 El procedimiento Path

La idea, propuesta por Savitch (1970), es aplicar divide y vencerás sobre la *longitud* del camino.

Se define $\mathsf{Path}(S_1, S_2, L)$ como el procedimiento que decide si existe una secuencia de *a lo más* L operadores que lleve de la configuración S_1 a la configuración S_2.

El problema original es $\mathsf{Path}(S_0, S^*, 2^n)$, lo cual es suficiente por el Teorema 4.3.

En lugar de hacer crecer el camino de uno en uno (como BFS), se parte en dos mitades de a lo más $\lceil L/2 \rceil$ pasos cada una, unidas por un *punto medio* S' (Kleinberg & Tardos, 2005, p. 541).

El procedimiento prueba las configuraciones posibles como puntos medios, una por una. Para cada candidata, primero comprueba si existe un camino desde S_1 hasta ella y después si existe uno desde ella hasta S_2. Si ambas respuestas son afirmativas, existe un camino entre las configuraciones originales. Al terminar cada llamada recursiva se reutiliza su memoria, conservando únicamente el resultado necesario.

Procedimiento $\mathsf{Path}(S_1, S_2, L)$ (Kleinberg & Tardos, 2005, p. 542)

Entrada: configuraciones S_1, S_2 y un entero L (potencia de 2).

si $L = 1$ **entonces**
  si $S_1 = S_2$ o hay un operador O que convierte S_1 en S_2 **entonces**
    devolver "sí"
  si no
    devolver "no"
  fin si
si no
  Enumerar todas las configuraciones S' con un contador de n bits.
  para cada S' **hacer**
    $x \leftarrow \mathsf{Path}(S_1, S', \lceil L/2 \rceil)$; borrar todo el trabajo intermedio salvo x.
    $y \leftarrow \mathsf{Path}(S', S_2, \lceil L/2 \rceil)$; borrar todo el trabajo intermedio salvo y.
    si $x =$ "sí" y $y =$ "sí" **entonces**
      devolver "sí"
    fin si
  fin para
  devolver "no"
fin si

Observación 4.5 (Adaptaciones menores). (1) El libro escribe el caso base solo con la condición "hay un operador que convierte S_1 en S_2"; aquí se agrega "$S_1 = S_2$" para que Path signifique literalmente "a lo más L pasos" (permite la secuencia vacía).

(2) El pseudocódigo calcula siempre x y y; si $x =$ "no" se puede omitir el cálculo de y sin afectar la correctitud ni el espacio. El ejemplo de la sección 7 usa esta optimización.

(3) Hay que recordar que L es un parámetro exponencialmente grande: $\log L = n$.

5 Demostración de que Planning pertenece a PSPACE

Para demostrar que Planning pertenece a PSPACE, necesitamos justificar dos propiedades del procedimiento Path: primero, que responde correctamente si existe un plan y, segundo, que utiliza una cantidad de memoria polinomial respecto al tamaño de la entrada.

5.1 Correctitud

Teorema 5.1 (Kleinberg y Tardos, 2005, (9.8), correctitud)**.** $\mathsf{Path}(S_1, S_2, L)$ *devuelve "sí" si y solo si existe una secuencia de a lo más L operadores que lleva de* S_1 *a* S_2*.*

Demostración. Por inducción sobre L (potencia de 2). Si $L = 1$, el algoritmo revisa explícitamente todos los operadores (y el caso $S_1 = S_2$), así que es correcto.

Sea $L > 1$.

($\Leftarrow$) Si existe una secuencia de longitud $L' \leq L$ de S_1 a S_2, sea S' la configuración que ocurre en la posición $\lceil L'/2 \rceil$ de la secuencia.

El tramo de S_1 a S' tiene longitud $\lceil L'/2 \rceil \leq \lceil L/2 \rceil$ y el tramo de S' a S_2 tiene longitud $L' - \lceil L'/2 \rceil \leq \lceil L/2 \rceil$.

Por hipótesis de inducción, $\mathsf{Path}(S_1, S', \lceil L/2 \rceil)$ y $\mathsf{Path}(S', S_2, \lceil L/2 \rceil)$ devuelven "sí"; como el ciclo prueba todas las configuraciones, en algún momento llega a este S' y el procedimiento devuelve "sí".

($\Rightarrow$) Si devuelve "sí", existe S' con ambas llamadas en "sí". Por hipótesis de inducción hay una secuencia de a lo más $\lceil L/2 \rceil$ operadores de S_1 a S' y otra de a lo más $\lceil L/2 \rceil$ de S' a S_2.

Al concatenarlas se obtiene una secuencia de a lo más L operadores de S_1 a S_2. □

5.2 Espacio polinomial

Teorema 5.2 (Kleinberg y Tardos, 2005, (9.8), espacio)**.** *El espacio que usa* $\mathsf{Path}(S_1, S_2, L)$ *es polinomial en n, k y* $\log L$*. En particular,* $\mathsf{Path}(S_0, S^*, 2^n)$ *usa espacio polinomial en n y k.*

Demostración. Sin contar el espacio de las llamadas recursivas, cada invocación usa una cantidad de espacio polinomial en n, k y $\log L$.

En cada momento hay una única llamada recursiva activa: el trabajo intermedio de las demás ya se borró, y de ellas solo se conserva un bit.

Por tanto, para un polinomio p, el espacio $S(n,k,L)$ satisface

$$S(n,k,L) \leq p(n,k,\log L) + S(n,k,\lceil L/2 \rceil), \qquad S(n,k,1) \leq p(n,k,1).$$

Al desenrollar la recurrencia durante $O(\log L)$ niveles se obtiene

$$S(n,k,L) = O(\log L \cdot p(n,k,\log L)),$$

que es polinomial en n, k y $\log L$. □

Cuenta explícita de la pila. Un marco de Path guarda: S_1, S_2 y S' (n bits cada uno), el contador que enumera las configuraciones (n bits), el exponente j con $L = 2^j$ ($O(\log n)$ bits) y los bits x, y.

Para decidir si un operador convierte S_1 en S_2 se recorren las listas de la entrada, con $O(n + \log k)$ bits adicionales.

Cada marco usa entonces $O(n + \log k)$ bits. Como L se divide entre 2 en cada nivel, desde 2^n hasta 1, la profundidad de la recursión es $n + 1$.

El espacio total es $O\big(n(n + \log k)\big)$ bits, más la entrada (que se lee pero no se copia). Como el tamaño de la entrada es al menos n y al menos proporcional a k, esta cota es polinomial en el tamaño de la entrada.

5.3 Tiempo de ejecución

El espacio se ahorra a costa de tiempo. Sea $N(L)$ el número de llamadas a Path que genera una llamada con parámetro L.

El ciclo recorre 2^n configuraciones y hace a lo más dos llamadas por cada una, así que

$$N(1) = 1, \qquad N(L) \leq 1 + 2^{n+1} N(L/2).$$

Con $a = 2^{n+1}$ y $L = 2^j$ resulta

$$N(2^j) \leq \sum_{i=0}^{j} a^i \leq 2a^j,$$

y por tanto

$$N(2^n) \;\leq\; 2 \cdot 2^{n(n+1)} \;=\; 2^{O(n^2)}.$$

Cada llamada cuesta tiempo polinomial, de modo que el tiempo total es $2^{O(n^2)}$: exponencial, pero con espacio polinomial.

Esto es lo opuesto a la programación dinámica, cuyo principio es guardar todo el trabajo intermedio para no recalcularlo; aquí se tira todo y se recalcula (Kleinberg & Tardos, 2005, p. 543).

5.4 Conclusión: pertenencia a PSPACE

Teorema 5.3. Planning $\in$ PSPACE.

Demostración. Por los Teoremas 4.3, 5.1 y 5.2: Path$(S_0, S^*, 2^n)$ decide la instancia y usa espacio polinomial. □

6 Demostración de la PSPACE-completitud de Planning

En la sección anterior demostramos que Planning $\in$ PSPACE. Sin embargo, para concluir que es PSPACE-completo también debemos demostrar que es PSPACE-difícil.

Esto requiere mostrar que cualquier problema de PSPACE puede reducirse en tiempo polinomial a una instancia de Planning.

6.1 PSPACE-dificultad y relación con QSAT

El libro observa que la PSPACE-dificultad se puede probar mediante una reducción desde QSAT, pero no desarrolla la prueba (Kleinberg & Tardos, 2005, p. 544).

QSAT pregunta si $\exists x_1 \forall x_2 \cdots \exists x_n \; \Phi(x_1, \ldots, x_n)$ es verdadera, con Φ una conjunción de cláusulas de tres términos; su PSPACE-completitud se debe a Stockmeyer y Meyer (1973) (Kleinberg & Tardos, 2005, p. 543).

Que la planeación sea difícil se entiende bien por esta conexión: QSAT modela la planeación de contingencias, en la que se pregunta si hay una decisión tal que *para toda* respuesta hay otra decisión, y así sucesivamente (Kleinberg & Tardos, 2005, p. 535).

La completitud de Planning fue establecida por Bylander (1994), quien muestra que decidir la existencia de un plan en STRIPS proposicional es PSPACE-completo en general; su demostración de dureza codifica una máquina de Turing mediante operadores STRIPS (véase la descripción en Edelkamp et al., 2026).

La dureza se demuestra con la reducción de la siguiente sección.

6.2 Reducción desde máquinas de Turing de espacio polinomial

Esta sección es un esbozo propio, que sigue la técnica estándar de simular un cómputo con condiciones y operadores; la prueba original y sus variantes están en Bylander (1994).

6.2.1 Idea

Sea $A \in$ PSPACE decidido por una máquina de Turing determinista $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{acc}})$ que usa a lo más $p(|w|)$ celdas con p un polinomio.

Dada una entrada w con $|w| = m$ y $s = p(m)$, se construye una instancia de Planning cuyas configuraciones representan las configuraciones instantáneas de M.

Cada transición de M se convierte en un operador. Entonces M acepta w si y solo si existe un plan.

Sin pérdida de generalidad se supone que, antes de aceptar, M borra la cinta (la llena de blancos $\sqcup$) y regresa la cabeza a la celda 1; esto no aumenta el espacio.

También sin pérdida de generalidad, M nunca intenta moverse a la izquierda desde la celda 1 ni a la derecha desde la celda s. Por eso basta definir operadores para las posiciones i con $1 \leq i + D \leq s$.

Así, la configuración aceptora es única, lo que permite expresarla como la meta *exacta* que exige el problema.

6.2.2 Construcción

- **Condiciones.** Q_q para cada estado $q \in Q$ ("la máquina está en el estado q"); H_i para $1 \le i \le s$ ("la cabeza está en la celda i"); y $\mathsf{S}_{i,a}$ para $1 \le i \le s$ y $a \in \Gamma$ ("la celda i contiene a a").

  En total, $n = |Q| + s + s|\Gamma|$ condiciones.

- **Operadores.** Por cada transición $\delta(q,a) = (q',b,D)$ con $D \in \{-1,+1\}$ y cada posición i tal que $1 \le i+D \le s$, un operador $O_{q,a,i}$ con

$$P = \{\mathsf{Q}_q, \mathsf{H}_i, \mathsf{S}_{i,a}\},$$

$$A = \{\mathsf{Q}_{q'}, \mathsf{H}_{i+D}, \mathsf{S}_{i,b}\},$$

$$D_{\text{el}} = \{\mathsf{Q}_q, \mathsf{H}_i, \mathsf{S}_{i,a}\}.$$

  Si alguna condición aparece a la vez en A y en la lista de eliminación (por ejemplo, si $q' = q$ o $b = a$), se quita de ambas listas; así quedan disjuntas y la condición sigue cumpliéndose porque estaba en P.

- **Configuración inicial.**

$$S_0 = \{\mathsf{Q}_{q_0}, \mathsf{H}_1\} \cup \{\mathsf{S}_{i,w_i} : 1 \le i \le m\} \cup \{\mathsf{S}_{i,\sqcup} : m < i \le s\}.$$

- **Meta.**

$$S^* = \{\mathsf{Q}_{q_{\text{acc}}}, \mathsf{H}_1\} \cup \{\mathsf{S}_{i,\sqcup} : 1 \le i \le s\}.$$

Ejemplo 6.1. La transición $\delta(q_1,0) = (q_2,1,+1)$ en la posición i da el operador con

$$P = \{\mathsf{Q}_{q_1}, \mathsf{H}_i, \mathsf{S}_{i,0}\},$$

adiciones

$$\{\mathsf{Q}_{q_2}, \mathsf{H}_{i+1}, \mathsf{S}_{i,1}\}$$

y eliminaciones

$$\{\mathsf{Q}_{q_1}, \mathsf{H}_i, \mathsf{S}_{i,0}\}.$$

Cambia de estado, escribe un 1 y mueve la cabeza una celda a la derecha.

6.2.3 Tiempo de construcción

Hay $n = |Q| + s(1+|\Gamma|)$ condiciones y a lo más $|\delta| \cdot s$ operadores, cada uno con a lo más tres elementos en cada lista.

Como M es fija y $s = p(m)$ es un polinomio, la instancia tiene tamaño polinomial en m y se construye en tiempo polinomial.

6.2.4 Correctitud

Invariante. En toda configuración alcanzable desde S_0 hay exactamente una condición Q_q, exactamente una H_i y, para cada celda i, exactamente una $\mathsf{S}_{i,a}$.

Es cierto en S_0 y cada operador lo preserva (elimina una de cada tipo y agrega una de cada tipo).

Por el invariante, cada configuración alcanzable codifica una configuración instantánea de M.

($\Rightarrow$) Si M acepta w, su cómputo es una sucesión de configuraciones instantáneas que no sale de las celdas $1..s$ y termina en la configuración aceptora.

Aplicando en orden los operadores $O_{q,a,i}$ de las transiciones usadas, se obtiene un plan de S_0 a S^*.

($\Leftarrow$) Si hay un plan, por el invariante en cada paso el único operador aplicable es el que corresponde a (q,a,i) de la configuración actual (la máquina es determinista), así que el plan reproduce el cómputo de M sobre w y alcanza S^*, es decir, la configuración aceptora.

Luego M acepta w.

Como M siempre se detiene, el plan existe si y solo si $w \in A$.

Se concluye que $A \leq_{\mathrm{P}} \mathsf{Planning}$ para todo $A \in \mathsf{PSPACE}$.

6.3 Conclusión: PSPACE-completitud

La reducción anterior demuestra que Planning es PSPACE-difícil, pues permite transformar cualquier problema de PSPACE en una instancia equivalente de planeación en tiempo polinomial.

Como también demostramos que Planning $\in$ PSPACE, se cumplen las dos condiciones necesarias para establecer su PSPACE-completitud.

Teorema 6.2 (Bylander, 1994)**.** Planning *es* PSPACE*-completo.*

7 Ejemplo de la ejecución del algoritmo

Se usa la instancia del contador de $n = 2$ bits del Teorema 4.1. Todos los resultados de esta sección fueron verificados con el programa del apéndice A.

7.1 La instancia

- Condiciones c_1, c_2 ($n = 2$, así que $L = 2^2 = 4$) y $2^2 = 4$ configuraciones.
- O_1: $P_1 = \varnothing$, $A_1 = \{c_1\}$, $D_1 = \varnothing$.
- O_2: $P_2 = \{c_1\}$, $A_2 = \{c_2\}$, $D_2 = \{c_1\}$.
- $S_0 = \varnothing$ y $S^* = \{c_1, c_2\}$; se escribe $M = \{c_1, c_2\}$ para la meta.

Se espera un plan de $2^2 - 1 = 3$ pasos. La pregunta es si $\mathsf{Path}(\varnothing, M, 4)$ devuelve "sí".

7.2 La grafica implícita $\mathscr{G}$

El algoritmo *nunca construye* esta grafica; se muestra solo para seguir su razonamiento.

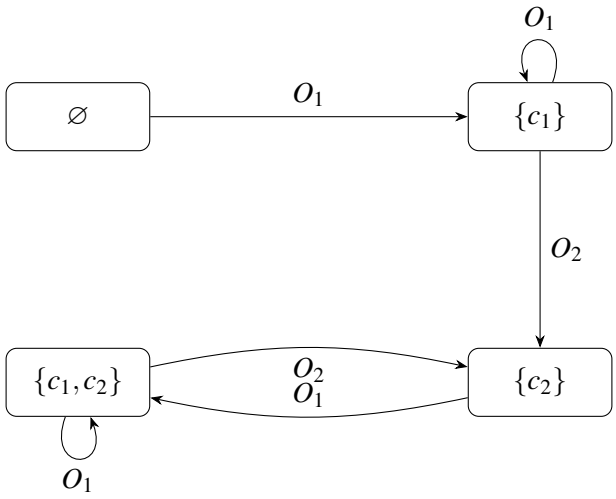


Figura 2: Grafica de configuraciones del contador de 2 bits. El plan de 3 pasos es $\varnothing \xrightarrow{O_1} \{c_1\} \xrightarrow{O_2} \{c_2\} \xrightarrow{O_1} \{c_1, c_2\}$.

El operador O_2 no es aplicable en $\varnothing$ ni en $\{c_2\}$ (falta c_1). El contador de $n = 2$ bits enumera las configuraciones en este orden: $\varnothing, \{c_1\}, \{c_2\}, \{c_1, c_2\}$ (el bit i del contador indica si c_i pertenece).

7.3 Ejecución de $\mathsf{Path}(\varnothing, M, 4)$

Como $L = 4 > 1$, se prueban los puntos medios S' en orden, con $\lceil L/2 \rceil = 2$:

S'	$x = \mathsf{Path}(\varnothing, S', 2)$	$y = \mathsf{Path}(S', M, 2)$	Resultado
$\varnothing$	sí	no	se descarta
$\{c_1\}$	sí	sí	**sí**

Como ya se encontró un S' con ambas respuestas en "sí", el ciclo se detiene y no se prueban $\{c_2\}$ ni M. A continuación se detalla cada fila (en cada subllamada, $\lceil 2/2 \rceil = 1$).

Fila $S' = \varnothing$. $x = \mathsf{Path}(\varnothing, \varnothing, 2)$ es "sí": con $S'' = \varnothing$, ambas llamadas $\mathsf{Path}(\varnothing, \varnothing, 1)$ son "sí" porque $S_1 = S_2$. Para $y = \mathsf{Path}(\varnothing, M, 2)$ se prueban los puntos medios S'':

S''	$\mathsf{Path}(\varnothing, S'', 1)$	$\mathsf{Path}(S'', M, 1)$
$\varnothing$	sí ($S_1 = S_2$)	no (O_1 lleva $\varnothing$ a $\{c_1\}$, no a M)
$\{c_1\}$	sí (O_1)	no ($O_1 \to \{c_1\}$, $O_2 \to \{c_2\}$)
$\{c_2\}$	no	no se evalúa
M	no	no se evalúa

Ningún S'' sirve, así que $y =$ "no". Es coherente: el camino más corto de $\varnothing$ a M mide 3 pasos, no 2.

Fila $S' = \{c_1\}$. $x = \mathsf{Path}(\varnothing, \{c_1\}, 2)$ es "sí": con $S'' = \varnothing$, $\mathsf{Path}(\varnothing, \varnothing, 1) =$ "sí" y $\mathsf{Path}(\varnothing, \{c_1\}, 1) =$ "sí" por O_1. Para $y = \mathsf{Path}(\{c_1\}, M, 2)$:

S''	$\mathsf{Path}(\{c_1\}, S'', 1)$	$\mathsf{Path}(S'', M, 1)$
$\varnothing$	no	no se evalúa
$\{c_1\}$	sí ($S_1 = S_2$)	no
$\{c_2\}$	sí (O_2)	sí (O_1: $\{c_2\} \to \{c_1, c_2\}$)

Con $S'' = \{c_2\}$ ambas llamadas dan "sí", así que $y =$ "sí".

Resultado. $\mathsf{Path}(\varnothing, M, 4) =$ "sí". De la ejecución se puede leer el plan $\varnothing \xrightarrow{O_1} \{c_1\} \xrightarrow{O_2} \{c_2\} \xrightarrow{O_1} \{c_1, c_2\}$, aunque el algoritmo solo devuelve el bit de respuesta. En total hubo 20 llamadas a Path con la optimización de omitir y cuando $x =$ "no" (23 sin ella).

7.4 Estado de la memoria

En el punto más profundo de la ejecución, la pila contiene tres marcos, no más:

1. $\mathsf{Path}(\varnothing, M, 4)$, con $S' = \{c_1\}$;
2. $\mathsf{Path}(\{c_1\}, M, 2)$, con $S'' = \{c_2\}$;
3. $\mathsf{Path}(\{c_2\}, M, 1)$.

Cuando termina una llamada se libera su marco y solo sobrevive un bit. En ningún momento se guardó el plan completo, que sería lo que exige tanto espacio en un enfoque con DFS o BFS.

7.5 ¿Qué pasa cuando n crece?

La Tabla 1 muestra el costo para el contador de n bits. La pila crece linealmente, mientras que el número de llamadas crece muy rápido (cota teórica $2^{O(n^2)}$).

n	Nodos de $\mathscr{G}$ (2^n)	Pasos del plan mínimo (2^n-1)	Marcos de pila ($n+1$)	Llamadas a Path
2	4	3	3	20
3	8	7	4	454
4	16	15	5	48 795
5	32	31	6	23 167 425

Tabla 1: Resultados medidos con el programa del apéndice A (con la optimización de omitir y). Para cada n, $\mathsf{Path}(\varnothing, \{c_1, \ldots, c_n\}, 2^n)$ devolvió "sí" y la longitud mínima, calculada con BFS, coincidió con $2^n - 1$.

Para $n = 60$ el plan mínimo tendría $2^{60} - 1 \approx 1.15 \times 10^{18}$ pasos, pero la pila tendría solo 61 marcos de $O(n)$ bits: del orden de unos cuantos miles de bits. Ese es el intercambio entre tiempo y espacio.

8 Enfoques para resolver el problema

Enfoque	Espacio	Tiempo	Comentario
BFS/DFS sobre $\mathscr{G}$	$2^{O(n)}$	$2^{O(n)}$ (poly en $\lvert\mathscr{G}\rvert$)	Es el más rápido, pero inviable en memoria para n grande.
Path (Savitch)	polinomial, $O(n(n + \log k))$ bits	$2^{O(n^2)}$	Prueba que Planning $\in$ PSPACE; no se usa en la práctica.
Planificadores prácticos	depende del dominio	sin garantía en el peor caso	Heurísticas y codificaciones; resuelven muchas instancias grandes.

Tabla 2: Comparación de enfoques.

8.1 Búsqueda explícita en $\mathscr{G}$

Construir $\mathscr{G}$ y recorrerlo con BFS o DFS toma tiempo y espacio $2^{O(n)}$ (por polinomios). Es el enfoque ingenuo de la sección 4.3.

8.2 Savitch: espacio polinomial a costa de tiempo

El procedimiento Path de la sección 4.4 es el enfoque teórico. Es un ejemplo de que la memoria puede reutilizarse de maneras que el tiempo no permite.

8.3 Planificadores prácticos

Los planificadores modernos no atacan el peor caso. Entre los enfoques más influyentes están la búsqueda heurística guiada por relajaciones del problema (el sistema FF; Hoffmann & Nebel, 2001), los planificadores

de búsqueda con heurísticas más elaboradas (Fast Downward; Helmert, 2006) y la planeación como satisfacibilidad, que codifica la existencia de un plan de longitud acotada como una fórmula SAT (Kautz & Selman, 1992). Una introducción completa a las técnicas de planeación automática es Ghallab et al. (2004). Que el peor caso sea PSPACE-completo explica por qué estos sistemas dependen de heurísticas y de la estructura de cada dominio, y por qué no pueden garantizar eficiencia para toda instancia.

9 Aplicaciones

9.1 Logística y operaciones de socorro

El ejemplo del libro: para coordinar una operación de ayuda en un desastre, quizá se necesiten veinte ambulancias en un lugar de alta montaña; para ello se requiere antes enviar diez quitanieves, lo cual exige combustible y tripulaciones, pero si el combustible se usa para los quitanieves puede no alcanzar para las ambulancias, etcétera (Kleinberg & Tardos, 2005, p. 533). Las técnicas de planeación automática se han usado a gran escala también en operaciones militares.

9.2 Vuelos con escalas y requisitos de visa

Una persona quiere viajar de Ciudad de México a Tokio. No basta con encontrar conexiones entre ciudades: algunos vuelos exigen documentos que solo se tramitan en ciertas ciudades. Las rutas y los requisitos son ficticios, y se omiten horarios y costos para concentrarnos en las condiciones y los operadores.

Instancia. Los vuelos disponibles son Ciudad de México $\to$ Los Ángeles $\to$ Madrid, Madrid $\leftrightarrow$ París y Madrid $\to$ Tokio. El vuelo a Los Ángeles exige visa estadounidense, que se tramita en Ciudad de México; el vuelo a Tokio exige visa japonesa, que solo se tramita en París. Todo vuelo y trámite requiere pasaporte, y los documentos se conservan durante el viaje.

Condición	Significado	Condición	Significado
c_1	Está en Ciudad de México	c_5	Está en Tokio
c_2	Está en Los Ángeles	c_6	Tiene pasaporte
c_3	Está en Madrid	c_7	Tiene visa estadounidense
c_4	Está en París	c_8	Tiene visa japonesa

Operador	Acción	P_i	A_i	D_i
O_1	Tramitar visa estadounidense	$\{c_1, c_6\}$	$\{c_7\}$	$\varnothing$
O_2	Volar a Los Ángeles	$\{c_1, c_6, c_7\}$	$\{c_2\}$	$\{c_1\}$
O_3	Volar de Los Ángeles a Madrid	$\{c_2, c_6\}$	$\{c_3\}$	$\{c_2\}$
O_4	Volar de Madrid a París	$\{c_3, c_6\}$	$\{c_4\}$	$\{c_3\}$
O_5	Tramitar visa japonesa	$\{c_4, c_6\}$	$\{c_8\}$	$\varnothing$
O_6	Volar de París a Madrid	$\{c_4, c_6\}$	$\{c_3\}$	$\{c_4\}$
O_7	Volar de Madrid a Tokio	$\{c_3, c_6, c_8\}$	$\{c_5\}$	$\{c_3\}$

El estado inicial es $S_0 = \{c_1, c_6\}$ y la meta es $S^* = \{c_5, c_6, c_7, c_8\}$. La meta incluye los documentos porque el problema exige alcanzar *exactamente* S^*.

Plan. La secuencia $O_1, \ldots, O_7$ recorre los estados:

Estado	Operador aplicado	Condiciones
S_0	— (inicial)	$\{c_1, c_6\}$
S_1	O_1	$\{c_1, c_6, c_7\}$
S_2	O_2	$\{c_2, c_6, c_7\}$
S_3	O_3	$\{c_3, c_6, c_7\}$
S_4	O_4	$\{c_4, c_6, c_7\}$
S_5	O_5	$\{c_4, c_6, c_7, c_8\}$
S_6	O_6	$\{c_3, c_6, c_7, c_8\}$
$S_7 = S^*$	O_7	$\{c_5, c_6, c_7, c_8\}$

El plan tiene siete acciones y es mínimo: ambas visas son necesarias y las rutas obligan a hacer esos cinco vuelos.

El viajero está en Madrid tanto en S_3 como en S_6, pero son estados distintos: en S_3 aún no tiene visa japonesa y O_7 no es aplicable. Por eso el recorrido Madrid–París–Madrid no es una vuelta inútil, y la grafica de planeación distingue ambos estados aunque correspondan a la misma ciudad.

Comprobación con Path. Con $L = 8$, una descomposición que funciona es la siguiente (con otro orden de enumeración, el algoritmo podría encontrar otro punto medio válido):

Llamada	Punto medio S'	Subllamadas
$\mathsf{Path}(S_0, S^*, 8)$	S_3	$\mathsf{Path}(S_0, S_3, 4)$, $\mathsf{Path}(S_3, S^*, 4)$
$\mathsf{Path}(S_0, S_3, 4)$	S_1	$\mathsf{Path}(S_0, S_1, 2)$, $\mathsf{Path}(S_1, S_3, 2)$
$\mathsf{Path}(S_3, S^*, 4)$	S_5	$\mathsf{Path}(S_3, S_5, 2)$, $\mathsf{Path}(S_5, S^*, 2)$

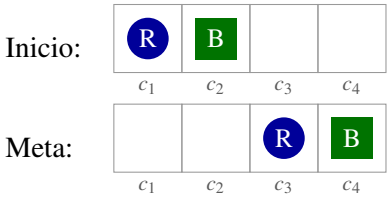
Cada llamada con $L = 2$ se resuelve con dos llamadas de $L = 1$, que verifican un operador del plan o, en el tramo $\mathsf{Path}(S_0, S_1, 2)$, la igualdad $S_0 = S_0$. Como Path pregunta por *a lo más* L pasos, el plan de siete acciones cabe en $L = 8$; en el algoritmo general se usaría $L = 2^8$. La pila nunca tiene más de cuatro marcos ($L = 8, 4, 2, 1$) y de cada mitad resuelta solo se conserva un bit.

Este ejemplo ilustra cómo las condiciones cambian las acciones disponibles, pero no demuestra por sí mismo la PSPACE-dificultad, que es una propiedad del problema general.

9.3 Robots de almacén: un desglose completo

Esta aplicación no aparece en el libro. Un robot debe empujar cajas por pasillos estrechos hasta posiciones de entrega, y algunos movimientos son irreversibles: una caja empujada contra una esquina ya no se puede recuperar. Es el mismo espíritu del juego Sokoban ("encargado de almacén" en japonés), que Culberson (1997) demostró PSPACE-completo, y del "problema del almacenista" (*warehouseman's problem*) de planeación de movimientos de varios objetos, que Hopcroft et al. (1984) demostraron PSPACE-difícil.

Instancia de juguete. Un pasillo de 4 celdas c_1, c_2, c_3, c_4. El robot (R) empieza en c_1, hay una caja (B) en c_2 y la meta es dejar la caja en c_4, con el robot en c_3.



Paso 1: condiciones. Para cada celda c: R@c (el robot está en c), B@c (hay una caja en c) y F@c (la celda c está libre). La condición F@c es positiva porque el modelo no admite negaciones. Hay $n = 12$ condiciones y por tanto 2^{12} configuraciones posibles (la mayoría físicamente absurdas; el modelo no lo sabe).

Paso 2: operadores.

Operador	Prerrequisitos	Agrega	Elimina
Move(a,b)	R@a, F@b	R@b, F@a	R@a, F@b
Push(a,b,c)	R@a, B@b, F@c	R@b, B@c, F@a	R@a, B@b, F@c

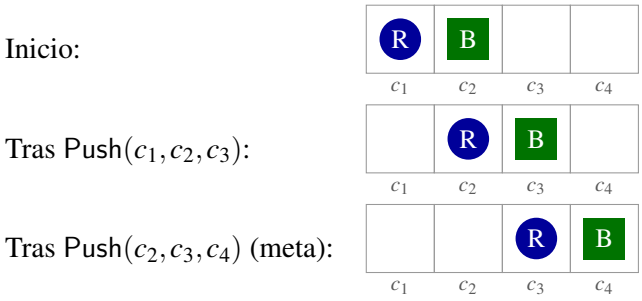
Move(a,b) se define para celdas adyacentes y Push(a,b,c) para tres celdas consecutivas. En el pasillo hay 6 operadores Move (en ambos sentidos) y 4 operadores Push, es decir, $k = 10$.

Paso 3: instancia y plan.

- $S_0 = \{\text{R@}c_1, \text{B@}c_2, \text{F@}c_3, \text{F@}c_4\}$.
- $S^* = \{\text{R@}c_3, \text{B@}c_4, \text{F@}c_1, \text{F@}c_2\}$.

Plan de dos pasos:

1. Push(c_1,c_2,c_3) lleva a $\{\text{R@}c_2, \text{B@}c_3, \text{F@}c_1, \text{F@}c_4\}$.
2. Push(c_2,c_3,c_4) lleva a $\{\text{R@}c_3, \text{B@}c_4, \text{F@}c_1, \text{F@}c_2\} = S^*$.



Una búsqueda en amplitud sobre esta instancia (verificada con el programa) confirma que 2 pasos es el plan mínimo.

Paso 4: por qué es difícil en general. Con cientos de celdas, varias cajas y callejones sin salida, la instancia sigue siendo una instancia de Planning con $n = \Theta(\text{celdas})$ condiciones: el espacio de configuraciones es exponencial, un plan puede ser largo y decidir si existe es PSPACE-completo para el caso general de Sokoban (Culberson, 1997). Los sistemas reales usan heurísticas y detección de bloqueos, pero no hay garantía de eficiencia para toda distribución de cajas.

9.4 Rompecabezas y juegos

El rompecabezas de quince y el cubo de Rubik se usan en inteligencia artificial como bancos de prueba de algoritmos de planeación (Kleinberg & Tardos, 2005, p. 534). Sokoban (sección anterior) es un ejemplo de rompecabezas cuya versión general es PSPACE-completa.

9.5 Misiones espaciales

La planeación automática se ha usado para controlar naves. El *Remote Agent*, de la NASA, integra planeación y calendarización con restricciones temporales, ejecución robusta y reconfiguración basada en modelos, y fue concebido como controlador a bordo de la misión Deep Space 1 (Muscettola et al., 1998). Cabe aclarar que su formalismo es más rico que el STRIPS proposicional de este reporte; se menciona como ejemplo del tipo de problema al que se aplican las técnicas de planeación.

10 Casos especiales y variantes

10.1 Longitud de plan acotada polinomialmente

Si se pide un plan de a lo más $q(n)$ pasos, con q un polinomio, el plan mismo es un certificado de longitud polinomial que se verifica simulando los operadores. Esta variante está en NP, a diferencia de la versión general, cuyos planes pueden ser exponenciales (Teorema 4.1).

10.2 Restricciones sobre los operadores

Bylander (1994) estudió qué ocurre al restringir los operadores:

- En general, decidir si existe un plan es PSPACE-completo.
- Con solo literales básicos, sigue siendo PSPACE-completo aun si cada operador tiene a lo más dos prerrequisitos y dos postcondiciones.
- Con fórmulas de Horn definidas, es PSPACE-completo aun con cero prerrequisitos y una postcondición.
- Un caso tratable: cada operador con prerrequisitos positivos y una sola postcondición (solo literales); el mundo de bloques, ligeramente modificado, es un subproblema.

Según el propio autor, se requieren restricciones extremadamente severas para garantizar tiempo polinomial o incluso NP-completitud, lo cual no es alentador para la planeación independiente del dominio.

10.3 Planeación de primer orden

Cuando los operadores se describen con predicados y variables (en lugar de proposiciones), la decidibilidad y la complejidad dependen de detalles como si se permiten símbolos de función, listas de eliminación o precondiciones negativas; Erol et al. (1995) establecen condiciones bajo las cuales la planeación es decidible o indecidible y cómo varía la complejidad en los casos decidibles.

10.4 Cuantificadores y juegos

QSAT y los juegos de dos jugadores, como *Competitive Facility Location*, comparten con Planning el hecho de ser PSPACE-completos y de resolverse en espacio polinomial reutilizando memoria. En QSAT la recurrencia es $S(n) \leq S(n-1) + p(n)$ (se elimina una variable por nivel) y en Planning es $S(L) \leq S(L/2) + p$ (se divide la longitud entre dos); en ambos casos de cada subproblema solo se guarda un bit (Kleinberg & Tardos, 2005, pp. 536–538, 542–543).

11 Conclusión

Planning pregunta si existe una secuencia de operadores que lleve de una configuración inicial a una meta. Su grafica implícita tiene 2^n nodos y sus planes mínimos pueden medir $2^n - 1$ pasos, por lo que el plan no es un certificado polinomial y Planning no es obviamente de NP. Sin embargo, el algoritmo de Savitch, basado en puntos medios y en reutilizar espacio, decide el problema en espacio $O(n(n + \log k))$ bits a costa de tiempo $2^{O(n^2)}$; por tanto, Planning $\in$ PSPACE. Mediante una reducción desde máquinas de Turing de espacio polinomial (Bylander, 1994), Planning es además PSPACE-difícil, así que es PSPACE-completo.

La lección general es que, para PSPACE, la memoria es el recurso que se puede reciclar. Si Planning tuviera un algoritmo de tiempo polinomial, entonces P $=$ PSPACE, lo que se considera muy improbable. En la práctica, las aplicaciones (logística, robótica de almacén, rompecabezas, misiones espaciales) recurren a heurísticas y a la estructura del dominio, no a garantías de peor caso.

Referencias

Bylander, T. (1994). The computational complexity of propositional STRIPS planning. *Artificial Intelligence, 69*(1–2), 165–204.

Culberson, J. (1997). *Sokoban is PSPACE-complete* (Technical Report TR 97-02). Department of Computing Science, University of Alberta.

Edelkamp, S., Fink, J., Gregor, P., Jonsson, A., & Nebel, B. (2026). *Intermediate results on the complexity of STRIPS*$^1_1$ [Preprint]. arXiv. https://arxiv.org/abs/2602.08708

Erol, K., Nau, D. S., & Subrahmanian, V. S. (1995). Complexity, decidability and undecidability results for domain-independent planning. *Artificial Intelligence, 76*(1–2), 75–88.

Fikes, R. E., & Nilsson, N. J. (1971). STRIPS: A new approach to the application of theorem proving to problem solving. *Artificial Intelligence, 2*(3–4), 189–208.

Ghallab, M., Nau, D., & Traverso, P. (2004). *Automated planning: Theory and practice*. Morgan Kaufmann.

Helmert, M. (2006). The Fast Downward planning system. *Journal of Artificial Intelligence Research, 26*, 191–246.

Hoffmann, J., & Nebel, B. (2001). The FF planning system: Fast plan generation through heuristic search. *Journal of Artificial Intelligence Research, 14*, 253–302.

Hopcroft, J. E., Schwartz, J. T., & Sharir, M. (1984). On the complexity of motion planning for multiple independent objects; PSPACE-hardness of the "warehouseman's problem". *The International Journal of Robotics Research, 3*(4), 76–88.

Kautz, H., & Selman, B. (1992). Planning as satisfiability. In *Proceedings of the 10th European Conference on Artificial Intelligence (ECAI-92)* (pp. 359–363). John Wiley & Sons.

Kleinberg, J., & Tardos, É. (2005). *Algorithm design*. Pearson.

Muscettola, N., Nayak, P. P., Pell, B., & Williams, B. C. (1998). Remote Agent: To boldly go where no AI system has gone before. *Artificial Intelligence, 103*(1–2), 5–47.

Savitch, W. J. (1970). Relationships between nondeterministic and deterministic tape complexities. *Journal of Computer and System Sciences, 4*(2), 177–192.

Sipser, M. (2013). *Introduction to the theory of computation* (3rd ed.). Cengage Learning.

Stockmeyer, L. J., & Meyer, A. R. (1973). Word problems requiring exponential time (preliminary report). In *Proceedings of the Fifth Annual ACM Symposium on Theory of Computing* (pp. 1–9). ACM.

A Código de verificación (Python)

El siguiente programa implementa Path (con la optimización de omitir y cuando x es "no") para el contador de n bits. Con él se verificaron el ejemplo de la sección 7 y la Tabla 1.

```
def contador(n):
    # Operadores O1..On del Teorema 9.6: (prerreq, adiciones, eliminaciones)
    return [(frozenset(range(1, i)), frozenset([i]), frozenset(range(1, i)))
            for i in range(1, n + 1)]

def un_paso(ops, c1, c2):
    return any(P <= c1 and (c1 - D) | A == c2 for P, A, D in ops)

def configuraciones(n):
    for q in range(2 ** n):                    # contador de n bits
        yield frozenset(i + 1 for i in range(n) if (q >> i) & 1)

def path(ops, n, c1, c2, L):
    if L == 1:
        return c1 == c2 or un_paso(ops, c1, c2)
    for c in configuraciones(n):
        if path(ops, n, c1, c, (L + 1) // 2) and path(ops, n, c, c2, (L + 1) // 2):
            return True
    return False

n = 2
print(path(contador(n), n, frozenset(), frozenset({1, 2}), 2 ** n))   # True
```